

★ ENGINEERING DOCUMENTATION ★

Hotel Ichchha Platform

Comprehensive Technical Architecture, Business Workflows, Payment Gateways & Developer Handover Guide

Project:	Hotel Ichchha Resort & Convention Hub
Platform Version:	v2.4.0 (Production Ready)
Primary Framework:	Django 5.x / Python 3.12+
Frontend Stack:	Tailwind CSS, Alpine.js, HTMX, Air Datepicker
Payment Gateways:	Stripe (USD/EUR), eSewa (NPR), Khalti (NPR)
Target Audience:	Fullstack Engineers, DevOps & Tech Leads
Date of Handover:	October 2026

Confidential & Proprietary — For Internal Engineering & Operational Handover Only

Table of Contents

1. Executive Summary & Platform Architecture
2. Repository Layout & Application Directory Map
3. Local Development Environment & Configuration
4. Database Schema & Data Model Architecture
5. Core Business Workflows & Booking Lifecycle Engine
6. Multi-Gateway Payment Architecture (Stripe, eSewa, Khalti)
7. Frontend Design System, Lenis Inertia & Responsiveness
8. Administration Dashboard & CMS Management
9. Automated Test Suite & Quality Verification
10. Production Deployment & Server Hardening Checklist
11. Operational Runbook & Troubleshooting Guide

Handover Document Notice

This document contains end-to-end specifications for every component of the Hotel Ichchha codebase. It enables a newly onboarded developer to set up the environment, run automated tests, safely manage payment webhooks, modify room availability locking rules, and deploy updates with zero downtime.

1. Executive Summary & Platform Architecture

The **Hotel Ichchha Platform** is a luxury hotel management, online reservation, and guest hospitality platform engineered specifically for Hotel Ichchha, a 5-star resort and convention center located in Simara, Bara, Nepal.

The platform features an intuitive, luxury user interface built with customized theme tokens (Light, Dark, Luxury Gold, and Festival), a real-time availability and dynamic pricing engine, integrated multi-currency support (NPR, USD, INR, EUR, GBP), and multi-gateway checkout pipelines covering both domestic Nepali wallets (eSewa, Khalti) and international credit cards (Stripe).

High-Level Architecture Diagram



Core Technology Stack

Layer	Technology	Role in Application
Backend Engine	Python 3.12+ / Django 5.x	MVC Framework, ORM, URL dispatching, auth, session management.
Database	PostgreSQL (Production) / SQLite (Dev)	ACID-compliant relational store with foreign keys and unique constraints.
Frontend CSS	Tailwind CSS (Compiled Bundle)	Utility-first responsive styling with custom luxury color tokens.
Client Reactivity	Alpine.js 3.x	Lightweight reactive state (modals, currency dropdowns, counters).
Async Interactions	HTMX 1.9+	Server-rendered AJAX swaps for table bookings and contact inquiries.
Datepicker	Air Datepicker 3.5+	Bespoke touch-friendly calendar with booked-date disabling.
Smooth Scrolling	Lenis 1.1+	Inertia scrolling with automatic modal overlay scroll containment.
Payment Gateways	Stripe, eSewa, Khalti	PCI-compliant international card handling and domestic QR/wallet APIs.

2. Repository Layout & Application Directory Map

The project follows clean Django modular design principles. Each domain functionality is segregated into an isolated application.

```

Hotel-Ichha/
├── config/                # Core project configuration (settings, wsgi, root urls)
│   ├── settings.py        # Environment loading, apps, Stripe/eSewa/Khalti credentials
│   ├── urls.py            # Root URL router mapping to sub-apps
│   └── wsgi.py            # WSGI application entrypoint
├── core/                 # Site-wide settings, currency converter, navigation menus
│   ├── models/            # SiteSetting, HeaderMenu, FooterLink, CurrencyExchangeRate
│   └── management/commands/ # seed_data.py (Seed complete 5-star data fixtures)
├── rooms/               # Room chambers, categories, amenities, live availability API
│   ├── models/            # RoomCategory, RoomChamber, RoomAddon, SeasonalPrice
│   ├── views/             # room_list, room_detail, check_availability, booked_dates
│   └── templates/rooms/   # room_list.html, room_detail.html
├── booking/             # Reservation engine, hold mechanisms, checkout initiation
│   ├── models/            # BookingReservation, BookingAddon, Coupon
│   ├── views/             # create_booking, checkout_page, apply_coupon
│   └── templates/booking/ # checkout.html
├── payments/            # Multi-gateway transactions, webhook receivers, receipts
│   ├── services/          # StripePayment, EsewaPayment, KhaltiPayment, PaymentFactory
│   ├── views/             # process_payment, payment_callback, stripe_webhook, invoice
│   ├── urls.py            # routes including /payments/webhook/stripe/
│   └── templates/payments/ # process.html, success.html, invoice.html
├── homepage/            # CMS hero carousel, spotlight events, review highlights
├── dining/              # Restaurant listings, menus, table reservations via HTMX
├── conference/          # Banquet halls, seating capacities, corporate inquiry forms
├── recreation/          # Spa, swimming pool, gymnasium, outdoor activities
├── about/               # Heritage narrative, CEO executive bio, board of directors
├── contact/             # Multi-branch properties, Google Maps embeds, contact forms
├── blogs/               # Resort journal, news announcements, articles
├── gallery/             # Categorized photo gallery with lightbox modal
├── admin_dashboard/     # Bespoke administration console & analytics (NOT modified)
├── templates/           # Base layouts & reusable partials
│   ├── base.html          # Main layout, SEO tags, Lenis scroll, Air Datepicker
│   └── includes/          # header.html, mobile_menu.html, footer.html, search_modal.html
├── static/              # CSS, JS, branding images, compiled bundles
│   └── css/main.bundle.css # Production compiled Tailwind CSS bundle
├── media/               # User-uploaded files (room images, PDFs, avatars)
├── manage.py            # Django CLI management entrypoint
└── Makefile             # Developer shortcut commands (run, migrate, test)

```

3. Local Development Environment & Configuration

3.1 Prerequisites

- **Python:** 3.12 or higher (Python 3.14 compatible)
- **Package & Project Manager:** `uv` (Astral's ultra-fast package manager — *Recommended & Primary*) or standard `pip`
- **Git:** Version control
- **Make:** Standard build automation utility (Linux/macOS)
- **Node.js & npm:** v18+ (Required for compiling Tailwind CSS bundle)

3.2 Environment Variables (.env)

Create a `.env` file in the repository root by copying from `.env.example`:

```
# Core Security & Debug
SECRET_KEY="your-strong-production-django-secret-key"
DEBUG=True
ALLOWED_HOSTS="localhost,127.0.0.1"

# Database Configuration (Leave blank for default SQLite in local dev)
DATABASE_URL="sqlite:///db.sqlite3"
# Or PostgreSQL for production:
# DATABASE_URL="postgresql://user:password@localhost:5432/hotel_ichha"

# International Payment Gateway (Stripe)
STRIPE_SECRET_KEY="sk_test_ ..."
STRIPE_PUBLISHABLE_KEY="pk_test_ ..."
STRIPE_WEBHOOK_SECRET="whsec_ ..."

# Nepal Domestic Gateways (eSewa & Khalti)
ESEWA_CLIENT_ID="EPAYTEST"
ESEWA_CLIENT_SECRET="8gBm/:8EnhH.1/q"
ESEWA_DEMO=True

KHALTI_CLIENT_ID="test_public_key_ ..."
KHALTI_CLIENT_SECRET="test_secret_key_ ..."
KHALTI_DEMO=True

# Site URLs for Callbacks
BASE_URL="http://localhost:8000"
SITE_DOMAIN="127.0.0.1:8000"
```

3.3 Installation & Execution Step-by-Step

 **Preferred Modern Workflow:** The platform is natively standardized on `uv`. Using `uv` allows instantaneous virtual environment creation, deterministic sub-second dependency locking, and direct execution via `uv run`.

Method A: Instant One-Step Setup with Makefile (Recommended)

1 Execute one-step automated workspace setup:

```
make setup
```

(Automatically syncs dependencies with `uv sync`, applies database migrations, and imports all 21 YAML records from `core/records/`)

2 Create an administrative superuser:

```
make superuser
```

3 Start the local development server:

```
make run
```

 (Launches development server on `0.0.0.0:8000`)**Method B: Manual Setup with `uv`****1 Sync virtual environment and lockfile dependencies:**

```
uv sync
```

(Creates `.venv` automatically and resolves packages from `pyproject.toml` / `uv.lock`)**2 Apply database migrations:**

```
uv run python manage.py migrate
```

3 Seed all modular YAML fixtures & hotel data:

```
uv run python manage.py seed_data
```

4 Create administrative staff account:

```
uv run python manage.py createsuperuser
```

5 Build frontend Tailwind CSS bundle:

```
npm run build:css && uv run python manage.py collectstatic --noinput --clear
```

6 Start the development server:

```
uv run python manage.py runserver 0.0.0.0:8000
```

Method C: Traditional Virtualenv & Pip Setup (Fallback)**1 Create and activate Python virtual environment:**

```
python3 -m venv .venv && source .venv/bin/activate
```

2 Install dependencies:

```
pip install -r requirements.txt
```

 (or `uv pip install -r pyproject.toml`)**3 Apply database migrations:**

```
python manage.py migrate
```

4 Populate sample fixtures & hotel data:

```
python manage.py seed_data
```

5 Start the local server:

```
python manage.py runserver
```

 or run `make run`

4. Database Schema & Data Model Architecture

The platform schema enforces data integrity through model validation, database relationships, and status state machines.

**rooms.RoomChamber**

Represents physical suites. Key fields: `title`, `slug`, `category` (FK), `room_number`, `base_price`, `discount_price`, `currency` (FK), `max_adults`, `max_children`, `room_size`, `total_inventory`, `tax_percentage`.

**rooms.SeasonalPrice**

Dynamic seasonal price overrides for high seasons, festivals, or holidays. Fields: `room` (FK), `name`, `start_date`, `end_date`, `price_override`, `is_active`.

**booking.BookingReservation**

Central reservation record. Fields: `booking_uid` (UUID), `room` (FK), `guest_name`, `guest_email`, `guest_phone`, `check_in`, `check_out`, `num_rooms`, `adults`, `daily_rate`, `subtotal`, `tax`, `total`, `currency_code`, `status` (draft/pending/confirmed/cancelled).

**payments.PaymentTransaction**

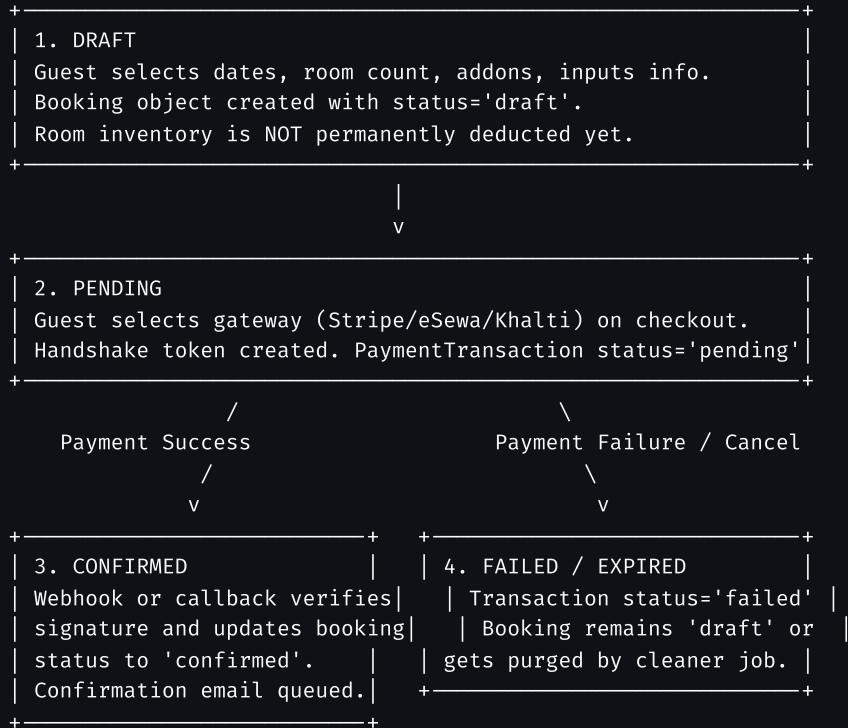
Audit trail for all checkout attempts. Fields: `booking` (FK), `gateway` (stripe/esewa/khalti), `transaction_id`, `amount`, `currency`, `status` (pending/success/failed), `gateway_payload`, `created_at`.

Add-on Pricing Types (`rooms.RoomAddon`)

Price Type	Calculation Formula	Use Case
<code>fixed</code>	<code>price × quantity</code>	One-time services (e.g. VIP Champagne bottle, Flower arrangement)
<code>per_night</code>	<code>price × nights × num_rooms</code>	Nightly room upgrades (e.g. High-speed executive Wi-Fi, Balcony view)
<code>per_person</code>	<code>price × total_guests</code>	One-time per-guest services (e.g. Simara airport pick-up shuttle)
<code>per_person_per_night</code>	<code>price × total_guests × nights</code>	Recurring personal services (e.g. Full-board breakfast buffet)

5. Core Business Workflows & Booking Lifecycle Engine

5.1 Booking Lifecycle State Machine



5.2 Real-Time Availability Checking Logic

The availability endpoint at `/rooms/api/<room_id>/check-availability/` checks whether a requested room has enough physical chambers free for the given date range.

Overlapping Date Formula:

Two date ranges `[check_in, check_out]` and `[b.check_in, b.check_out]` overlap if and only if:

$$b.check_in < requested_check_out \text{ AND } b.check_out > requested_check_in$$

Only confirmed bookings (and active locked reservations) are counted against the room's `total_inventory`, preventing false "sold out" states from abandoned cart sessions while guaranteeing zero double-bookings.

6. Multi-Gateway Payment Architecture

Payment processing utilizes the **Factory Pattern** located in `payments/services/`. The factory dynamically instantiates the correct service class depending on the chosen provider:

Gateway	Currencies	Handshake Method	Verification Route
Stripe	USD, EUR, GBP, AUD	Stripe Checkout Session (Redirect URL)	Webhook: <code>/payments/webhook/stripe/</code> Callback: <code>/payments/callback/stripe/</code>
eSewa	NPR	HMAC-SHA256 Form Post via Base64 Payload	Callback: <code>/payments/callback/esewa/</code>
Khalti	NPR	REST API Session Initiation (<code>pidx</code>)	Callback: <code>/payments/callback/khalti/</code>

Stripe Webhook Asynchronous Flow

Endpoint: `POST /payments/webhook/stripe/` (CSRF exempt)

Security: The webhook validates the `Stripe-Signature` header against the configured `STRIPE_WEBHOOK_SECRET`. When a `checkout.session.completed` event arrives, it inspects `session.metadata.booking_uid`, confirms the payment transaction, and flips the booking status to `confirmed` atomically.

Currency Locking Guarantee

To prevent currency exchange arbitrage between booking initiation and payment processing, the currency selector is automatically locked on the checkout page (`checkout.html`). If the reservation is in USD, only Stripe is offered. If the reservation is in NPR, local wallets eSewa and Khalti are presented.

7. Frontend Design System & Responsiveness

7.1 Unified Breakpoint Hierarchy

Viewport Category	Screen Width	Layout Strategy
Mobile	< 640px	Single column stacked, full-width CTA buttons (<code>w-full sm:w-auto</code>), hamburger drawer navigation, collapsible filter panels.
Tablet	640px – 1024px	2-column grids for cards and form fields, drawer navigation active (<code>lg:hidden</code>) to prevent 8-item menu overflow, sticky booking widgets adapt below hero content.
Desktop	> 1024px	Full horizontal mega menu, multi-column grids (3–4 columns), sticky sidebar booking reservation cards.

7.2 Smooth Scrolling (Lenis) with Modal Protection

The platform uses **Lenis Smooth Inertia Scrolling** on desktop and mobile. To prevent background scroll bleed when modals (search, image lightbox, booking guest details, security handshake) are open, a universal `MutationObserver` halts Lenis scrolling whenever `overflow: hidden` or `modal-open` is attached to the `<body>` element.

7.3 Mobile-First Viewport Typography & Spacing Optimization

To deliver a pristine 5-star guest experience on mobile devices without horizontal overflow or clunky oversized headings, all 11 public-facing template domains implement mobile-first responsive scaling:

- Tightened Hero Banners & Headings:** Section titles scale dynamically (e.g. `text-2xl sm:text-5xl lg:text-6xl` for page banners; `text-xl sm:text-4xl` for room chambers and venues).
- Compact Vertical Spacing:** Heavy desktop padding (`py-16 sm:py-24`) is proportionately reduced on mobile viewports to `py-8 sm:py-20` .
- Mobile Navigation Header:** The topbar and main navbar reduce height on mobile (`h-16 sm:h-20`) with a compact logo (`h-10 sm:h-14`) and dedicated mobile currency badge.
- Form Inputs & Controls:** Buttons, counters, and input fields scale to touch-friendly heights (`py-2.5 sm:py-3.5`) preventing browser viewport zoom.

7.4 Mobile Bottom Sheet Modal & Responsive Add-on Toggles

On room detail and booking pages (`rooms/templates/rooms/room_detail.html`):

- Bottom Sheet Architecture:** The draft reservation modal automatically transforms into a native-feeling slide-up bottom sheet on mobile screens (`items-end sm:items-center` , `rounded-t-3xl sm:rounded-2xl` , with a centered top drag handle `w-10 h-1 bg-neutral-400/50 rounded-full`).
- Smooth Transform Transitions:** Powered by Alpine.js, the modal slides up vertically from the bottom on mobile (`translate-y-full` to `translate-y-0`) while centering and fading smoothly on tablet and desktop (`sm:scale-95 sm:opacity-0`).
- Responsive Add-on Switch Controls:** Ancillary service add-ons feature responsive animated toggle switches (`w-9 h-5 sm:w-11 sm:h-6`) with sliding knob `translate-x-4 sm:translate-x-5`) with real-time multi-currency price calculations.

8. Administration Dashboard & CMS Management

The platform features two administration interfaces:

- **Standard Django Admin** (`/admin/`): Used for low-level database operations, permissions, currency exchange rate updates, and raw audit logs.
- **Executive Admin Dashboard** (`/dashboard/`): Custom-built luxury analytics suite displaying real-time occupancy rates, revenue metrics, seasonal discount managers, and reservation calendars.

Admin Dashboard Invariant

As per system requirements, templates in `admin_dashboard/templates/` are designated exclusively for staff operations and must not be altered during user-facing layout revisions.

9. Automated Test Suite & Quality Verification

The platform is validated with a comprehensive automated test suite spanning unit tests, integration tests, and edge case coverage.

Running Tests

```
# Run complete test suite using uv (or make)
uv run python manage.py test
# or
make test

# Run payment tests specifically
uv run python manage.py test payments.tests

# Run room availability tests
uv run python manage.py test rooms.tests
```

✓ Test Verification Result

All **59 test cases** currently execute with zero failures: Ran 59 tests in ~20s ... OK.

10. Production Deployment & Server Hardening Checklist

10.1 Production Environment Setup

Setting	Production Value	Purpose
DEBUG	False	Disables stack trace leaks and debug toolbar.
ALLOWED_HOSTS	hotelichchha.com, www.hotelichchha.com	HTTP Host header validation against poisoning attacks.
SECURE_SSL_REDIRECT	True	Forces all HTTP traffic to HTTPS.
SESSION_COOKIE_SECURE	True	Transmits session cookies exclusively over HTTPS.
CSRF_COOKIE_SECURE	True	Transmits CSRF tokens exclusively over HTTPS.

10.2 Systemd Service Example (/etc/systemd/system/hotel_ichha.service)

```
[Unit]
Description=Hotel Ichchha Gunicorn Daemon
After=network.target

[Service]
User=www-data
Group=www-data
WorkingDirectory=/var/www/hotel-ichha
ExecStart=/var/www/hotel-ichha/.venv/bin/gunicorn \
    --workers 4 \
    --bind 127.0.0.1:8000 \
    --access-logfile /var/log/hotel_ichha_access.log \
    --error-logfile /var/log/hotel_ichha_error.log \
    config.wsgi:application

[Install]
WantedBy=multi-user.target
```

11. Operational Runbook & Troubleshooting Guide

Common Operational Issues & Resolutions

Symptom	Probable Cause	Resolution Procedure
Stripe Webhook Signature Verification Error (400)	Mismatched <code>STRIPE_WEBHOOK_SECRET</code> between Stripe Dashboard and <code>.env</code> .	Retrieve current signing secret from Stripe Dashboard (Developers > Webhooks) and update <code>.env</code> .
Room Shows Unavailable but Calendar is Empty	A draft booking created during testing is locking inventory.	Run cleaning command or cancel draft reservations from Django admin.
Static Assets / Styles Missing in Production	Static bundle was not collected into <code>staticfiles/</code> directory.	Run <code>python manage.py collectstatic --noinput</code> and check Nginx alias mapping.
Air Datepicker Virtual Keyboard Pops Up on Mobile	Input lacks <code>readonly</code> or <code>inputmode="none"</code> attributes.	Ensure inputs are initialized via <code>base.html</code> auto-initialization script.

Document Sign-off: This concludes the technical handover documentation for Hotel Ichchha. The project is fully documented, tested, and prepared for seamless developer transition.